Part A: Computer Vision Challenge

4th International Autonomous Greenhouse Challenge

Version: 15-03-2024



In 2024, we will organize the 4th International Challenge to sustainably grow vegetables in greenhouses by different international teams. Teams will develop algorithms for a fully autonomous control of crop production. Their algorithms will focus on the sustainable use of resources. This year's crop will be dwarf tomatoes. A crop which is highly suitable for automation and for growing in urban environments.

This document describes the details of the *Part A Computer Vision* part of the online challenge.

Table of Contents

1 Preface ..... - 2 -

2 Imaging process ..... - 3 -

2.1 Example images ..... - 3 -

2.2 Plant measurements ..... - 3 -

3 Organization of the image data ..... - 5 -

3.1 Reconstructing 3d imaging ..... - 5 -

3.2 Dataset split: train, validation(s), test ..... - 5 -

3.3 Folder structure ..... - 6 -

4 Error metric ..... - 7 -

4.1 Example Python implementation ..... - 7 -

5 Online challenge setup ..... - 8 -

5.1 Intermediate ranking on the validation sets ..... - 8 -

5.2 Final ranking on test set ..... - 8 -

5.3 Hackathon event: Results on Computer Vision Challenge combined with other results - 8 -

6 Disclaimer ..... - 8 -

1 Preface

The 4th international challenge on autonomous greenhouses aims at having a fully automated control of growing dwarf tomatoes. This can potentially replace labour-intensive traditional high-wire tomato production. In modern greenhouses, the level of automation for any crop production is already high. Climate control and irrigation in modern greenhouses are in general automated, however, decisions on setpoints are in general made by the grower. Spacing of plants in pots as they become larger can be done by robots or some well-designed mechanical system, however, also here decisions are made by humans until now.

During greenhouse production decisions on what climate and irrigation to apply, when spacing at what plant distance is performed, and whether the tomato plants are ready for harvest, are made by human operators. Based on their knowledge and experience they make decisions to influence growing process and to allow for optimal resource use. These decisions are based on the observed growing process. It means that the current crop growth must be compared with an expected future growth to determine if the growing process is still on track. To do so, observations are generally made by visual inspection by an experienced human grower. This is a time-consuming process, requires loads of experience and is subjective. To assist or replace the human eye by an automated decision process, computer vision could be used to quantify the crop growth. This automatic assessment of crop status based on images is the goal of this online computer vision challenge. Teams are challenged to develop software that performs this conversion of images to numerical crop parameters.

This document describes the dataset, how it will be provided to the teams over the course of the challenge, and how teams will be scored.

2 Imaging process

The dataset consists of images of dwarf tomato plants. The images are taken with a RGBD camera under defined conditions and contain images of individual dwarf tomato plants of four different varieties, in different growth stages and grown in different lighting and irrigation (EC) conditions. On fixed dates, a fixed number of plants from each of the four cultivars is removed from the greenhouse, imaged, and subsequently (destructively) measured.

Each image is connected to information on the manually measured plant traits: plant height, plant fresh weight, leaf area and the number of fully red fruits. Teams use the images to develop a computer vision algorithm. This algorithm will have to be able to estimate the plant traits of a set of unseen dwarf tomato plant images, provided after the preparation phase, in a limited timeframe.

2.1 Example images

Images were taken with one Oak-D S2 POE camera and the dataset includes the collected RGB and Depth images. The camera is mounted about 90 cm above the ground, facing downwards. An example image is shown in Figure 1.



Figure 1. Example image *train/rgb*/C_b33d9a15.*png*

The dataset contains 4 different cultivars, named A, B, C and D. The cultivar is specified as prefix in the filename, i.e., Figure 1 (*C_b33d9a15.png*) shows an image of cultivar C. Examples of all cultivars are shown in *Table 1.*

2.2 Plant measurements

The following plant parameters are measured, and the goal of the challenge is to estimate these parameters based on the RGBD imaging.

Fresh weight of the plant (`fw_plant`) in grams per plant: The fresh weight of the plant, including stem, leaves, flowers, and fruits.

Height (`height`) in cm: The height of the highest part of the plant, measured vertically from the "substrate level", in cm. Cultivars A, C and D grow in a pot filled with a sustainable natural substrate, with a pot height of 10 cm, the "substrate level" is lower than the pot height. Cultivar B grows in a stonewool substrate block covered with a plastic foil, and the height of the substrate block is 7 cm, equal to the "substrate level".

Leaf area (`leaf_area`) in cm2: For the destructive measurements, leaves are cut from the stem and their surface is measured. The sum of all leaf areas is captured in this trait.

Number of red fruits (`number_of_red_fruits`) as count: The number of ripe tomatoes counted, where ripeness is based on their color. A distinction was made between red, orange, and green fruits. Only ripe, fully red tomatoes are counted.

Note 1: Missing values are recorded as `NaN` in the ground truth json file, and only occur a few times. In the evaluation, the predictions for which the ground truth is missing are ignored (Section **Error! Reference source not found.**).

Note 2: All ground truth measurements are registered manually and can therefore contain errors. Height, and number of fruits that is red, are inspected by hand/eye, so these are observer dependent to some degree. The goal of this challenge is to predict these manually measured values, even if you might disagree with the observation.

Table 1. Examples of the four cultivars included in the data set. These example images show plants in different stages.

Cultivar A	**Cultivar B**
 A_98ec0fae (cropped)	 B_07ea549e (cropped). Note: This cultivar grows in stonewool substrate.
Cultivar C	**Cultivar D**
 C_b33d9a15 (cropped)	 D_ae7f94a8 (cropped)

3 Organization of the image data

The RGB and depth images can be linked together by their filename, as well as their ground truth measurements in `ground_truth.json`. For example, for plant `C_0c172001` (which is cultivar C, as its prefix) the following information is available:

- RGB image: `train/rgb/C_b33d9a15.png`
- Depth images:
  - `train/depth/C_b33d9a15_irL.png`
  - `train/depth/C_b33d9a15_irR.png`
  - `train/depth/C_b33d9a15_depth.png`
- Ground truth in `train/ground_truth_train.json`:

```
"C_b33d9a15": {
    "height": 26.0,
    "fw_plant": 267.23,
    "leaf_area": 1200.08,
    "number_of_red_fruits": 8.0
},
```

3.1 Reconstructing 3d imaging

Images were taken with one Oak-D S2 POE camera and the dataset includes the collected RGB and Depth data. For each plant, there is an RGB image, a left and right IR image, and a depth image, which can be linked together by their filename. All camera parameters can be found in the intrinsics json-file. Both the intrinsics and extrinsics are given. The intrinsics for the color and depth image are the same, as the depth images are aligned with the color images. This means that the extrinsic information can be neglected. The depth images can be converted to point clouds by using these intrinsics and the depth scale. Teams are free to improve the depth imaging by using the provided infrared images. An example script on how to convert depth images to point clouds can be found at https://github.com/WURgreenhouse/point-cloud-demo.

3.2 Dataset split: train, validation(s), test

The dataset is released over the course of the challenge. In the first phase, a **training set with ground truth measurements** is released. Teams can use this set to develop their solution.

During the online challenge, on a 2-weekly basis, 3 **validation sets** are released. These sets are used to measure the teams' progress, but do not count for the final ranking. If your team provided a prediction for a validation set, you will receive the ground truths for that set one day later. This way, over the course of the challenge, the set of images with ground truths that you can use for model training increases.

At the end of the online challenge, a **test set** is provided. The results on the test set determine your final ranking. The final ranking will not be made public until the Hackathon event.

In total, 479 dwarf tomato plants have been imaged. The train set consists of ~50% of the images, the validation set of ~30% of the data, split into 3 subsets of ~10% of the images each. The final test set consists of the remaining ~20% of images. The splits of train, validation and test have been stratified on cultivar, imagining date (representing different growth stages), and growing strategy. Of these plant parameters, only the cultivar is provided to the teams, while the date and strategy are not disclosed. The subsequent split of the validation set into 3 subsets is only stratified on cultivar and date, because of the small subset sizes.

Table 2. Description of the different data sets. The full data set, including test ground truth, will be made publicly available after the Hackathon event. All dates and times are local Dutch time, Central European Summer Time (CEST).

Set	**Release date**	**Release date of ground truth**	**Number of images**
Training set	Tuesday, April 2 14:00	Tuesday, April 2 15:00	268
Validation subset 1	Tuesday, April 16 14:00	Wednesday, April 17 15:00	39
Validation subset 2	Tuesday, April 30 14:00	Wednesday, May 1 15:00	38
Validation subset 3	Tuesday, May 14 14:00	Wednesday, May 15 15:00	38
Test set	Friday, May 31 14:00	*after Hackathon event*	96

3.3 Folder structure

The sets in the dataset are structured as follows.

```
.
├── train [4.17 GB Zipped]
│   ├── ReadMe.md
│   ├── demo_response_train.txt
│   ├── demo_submission_train.json
│   ├── ground_truth_train.json
│   ├── oak-d-s2-poe_conf.json
│   ├── depth  [804 image files]
│   └── rgb  [268 image files]
├── validation1 [608 MB Zipped]
│   ├── depth  [117 image files]
│   └── rgb  [39 image files]
├── validation2 [590 MB Zipped]
│   ├── depth  [114 image files]
│   └── rgb  [38 image files]
├── validation3 [590 MB Zipped]
│   ├── depth  [114 image files]
│   └── rgb  [38 image files]
├── test [1,49 GB Zipped]
│   ├── depth  [288 image files]
│   └── rgb  [96 image files]
└── validation_truths
    ├── ground_truth_validation1.json
    ├── ground_truth_validation2.json
    └── ground_truth_validation3.json
```

The train set contains additional files that are applicable to the entire dataset:

- `ReadMe.md` contains roughly the information that is provided in this document as well.
- The file `oak-d-s2-poe_conf.json` contains information on the camera configuration, which remains constant for all sets.
- The files `demo_repsonse_train.txt` and `demo_submission_train.json` can be used to validate your implementation of the score function, as described below.

4 Error metric

The error metric we use is the average root mean squared relative error (RMSRE) for the different plant traits. We look at the relative error to account for different magnitudes of traits during the growing period, and to match the unit of the errors of different traits. We increment the ground truth and prediction with 1 to prevent division by zero errors. This is required for the number of red fruits, as this can take the value 0. For consistency, we do include this +1 for all other traits as well. Thus, the overall error is calculated with the following formula:

$$\text{Error}(g, p) = \frac{1}{T}\sum_{t=1}^{T}\sqrt{\frac{1}{N_t}\sum_{i=1}^{N_t}\left(\frac{p_i^t - g_i^t}{g_i^t + 1}\right)^2}$$

Here,

- $t = 1, \ldots, T$ with $T = 4$ represent the plants traits that need to be predicted. (`fw_plant`, `leaf_area`, `height`, `number_of_red_fruits`)
- $i = 1, \ldots, N_t$ represent the number of predictions for trait t in this set.
- g^t represents the vector of ground truth measurements for trait t.
- p^t represents the corresponding vector of your teams' predictions for that trait.

The lower the error, the better.

There are a few missing values in the dataset. These are stored as `NaN` in the json file. You should provide a prediction for all images and all traits, but the prediction for missing traits is ignored, similar to how `np.nanmean` handles these (see example below).

A submission is only valid if predictions are provided for all traits of all the images in the current set, including the traits that have a `NaN` (you could of course not foresee whether an image will have a missing value). If your result contains a `NaN`, is missing any predictions for any image or trait, or can for any other reasons not be processed, your submission is invalid, and will not be accepted.

4.1 Example Python implementation

A simple python implementation (without proper validation or exception handling) for computing the error is based on the json structure in `demo_submission_train.json` and `ground_truth_train.json` is:

```python
def calculate_error(pred: dict, truth: dict) -> float:
    error = 0
    for trait in ["height", "fw_plant", "leaf_area", "number_of_red_fruits"]:
        diff = [((pred[i][trait] - truth[i][trait]) / (truth[i][trait] + 1))**2 for i in truth]
        error += np.sqrt(np.nanmean(diff))
    return error / 4
```

To validate your implementation of the error calculation, you can use the files `demo_submission_train.json` and `ground_truth_train.json`, which should give the error value in `demo_repsonse_train.txt`.

5 Online challenge setup

The team captain will get login credentials to the portal, https://greenhousechallenge.azurewebsites.net/competition, from which the different sets are accessable, as specified in Table 2. The train set is available for download the entire duration of the challenge.

Your algorithm should give an output in the same structure as `demo_submission_train.json`. To test whether your output format is correct, and whether the upload works for your team, you can use the demo upload in the portal.

5.1 Intermediate ranking on the validation sets

Three times during the challenge, a validation subset will be released, according to the schedule described in Table 2. From the moment these datasets are available. Your team has 24 hours to start the download of the dataset. As sufficient processing speed is one of the qualities of your images processing algorithm, you should upload your prediction to the portal again (in json format) within 15 minutes after you started the download.

Each team that provided a valid prediction, will get access to the ground truth data of that set as reward, one day later. This way, you can extend your train dataset throughout the challenge.

Based on the errors of each team on the intermediate results, an intermediate ranking will be made, that will be publicly available at: https://greenhousechallenge.azurewebsites.net/. The team with the lowest prediction error will be ranked first. The intermediate errors are to measure teams' progress, but do not contribute to the final ranking.

5.2 Final ranking on test set

The final ranking is calculated on the prediction errors for test set. The test set is released on May 31, 14:00 CEST, and a result must be submitted before 14:30 CEST. Note that for the final testing procedure, you do not have 24 hours, but a prediction result must be submitted before the deadline. The rankings or scores are not announced directly but will be announced at the Hackathon event 6-7 June 2024 @WUR in Bleiswijk, The Netherlands. The team with the lowest error will be ranked first for this part of the Online Challenge.

5.3 Hackathon event: Results on Computer Vision Challenge combined with other results

During the Hackathon Event June 6-7th @WUR in Bleiswijk, The Netherlands, the ranking of the teams in the Computer vision part of the challenge (a.) and the ranking of the teams in the Climate and crop control part of the challenge (b.) will be announced. An additional task has to be performed during the Hackathon event (c.) and teams have to pitch their approach in front of an international jury (d.). All results are combined into an overall performance, according to the percentages described in the general rules and regulations.

6 Disclaimer

This event is organised by Wageningen University & Research and is sponsored by Tencent. We also thank the following companies: Vreugdenhill for providing seeds, Quantified for providing substrate sensors, Biobest for being partner in pest detection and biological control, Fluence for providing LED lamps, Lensli Substrates for proving sustainable substrates and Pöppelmann for providing recycled pots for the pre-trial and Certhon for providing images and data.

This challenge is organized with care and dedication, but we cannot completely rule out technical disruptions (e.g. malfunctioning mail servers or download link servers). In case teams' contributions clearly suffer from such issues, a suitable solution will be found.
Autonomous Greenhouse Challenge organizers can be contacted via: autonomousgreenhouses@wur.nl only.


Tencent 腾讯
biobest
SUSTAINABLE CROP MANAGEMENT
QUANTIFIED
sensor technology
VREUGDENHIL
Breeding & Seeds
certhon
FLUENCE
LENSLI
SUBSTRATES
PÖPPELMANN